

Bir Lama İçin Engeller

Bir lama, And Dağları Platosu'nda seyahat etmek istiyor. Elinde $N \times M$ kare hücreden oluşan bir ızgara biçiminde bir plato haritası var. Haritanın satırları yukarıdan aşağıya 0 ile $N - 1$ arasında numaralandırılmış ve sütunları soldan sağa 0 ile $M - 1$ arasında numaralandırılmıştır. Haritanın i satırı ve j sütunundaki hücre $(0 \leq i < N, 0 \leq j < M)$ (i, j) ile gösterilir.

Lama, platonun iklimini inceledi ve haritanın her satırındaki tüm hücrelerin aynı **sıcaklığa** ve haritanın her sütunundaki tüm hücrelerin aynı **neme** sahip olduğunu keşfetti. Lama size sırasıyla N ve M uzunluğunda iki tam sayı dizisi T ve H verdi. Burada $T[i]$ $(0 \leq i < N)$ i satırındaki hücrelerin sıcaklığını, $H[j]$ $(0 \leq j < M)$ ise j sütunundaki hücrelerin nemini göstermektedir.

Lama platonun florasını da incelemiş ve bir (i, j) hücresinin ancak ve ancak sıcaklığı neminden büyükse ($T[i] > H[j]$ şartı sağlanıyorsa) **bitki örtüsünden arınmış** olduğunu fark etmiştir.

Lama, platoyu yalnızca **geçerli yolları** izleyerek geçebilir. Geçerli bir yol, aşağıdaki koşulları sağlayan farklı hücrelerden oluşan bir dizidir:

- Yoldaki her bir ardışık hücre çifti ortak bir kenarı paylaşır.
- Yol üzerindeki tüm hücreler bitki örtüsünden arındırılmıştır.

Göreviniz Q adet soruyu cevaplamaktır. Her soru için size dört tam sayı verilir: L, R, S ve D . Şu şartları sağlayan geçerli bir yolun var olup olmadığını belirlemelisiniz:

- Yol $(0, S)$ hücresinde başlar ve $(0, D)$ hücresinde biter.
- Yoldaki tüm hücreler L ile R arasındaki (sınırlar dahil) sütunlar arasında yer alır.

Hem $(0, S)$ hem de $(0, D)$ hücresinin bitki örtüsünden arınmış olduğu garanti edilir.

İmplementasyon Detayları

Uygulamanız gereken ilk prosedür şudur:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : her satırdaki sıcaklığı belirten N uzunluğunda bir dizi.
- H : her sütundaki nemi belirten M uzunluğunda bir dizi.
- Bu prosedür, `can_reach` çağrılarında önce her test durumu için tam olarak bir kez çağrılır.

Uygulamanız gereken ikinci prosedür şudur:

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : Bir soruyu tanımlayan tam sayılar.
- This procedure is called Q times for each test case. Bu prosedür her test durumu için Q kez çağrılır.

Bu prosedür, yalnızca $(0, S)$ hücresinden $(0, D)$ hücresine tüm hücreleri L ile R arasındaki sütunlarda (sınırlar dahil) bulunan geçerli bir yol varsa `true` değerini dönmelidir.

Kısıtlar

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq i < N$ şartını sağlayan her i için $0 \leq T[i] \leq 10^9$ 'dur.
- $0 \leq j < M$ şartını sağlayan her j için $0 \leq H[j] \leq 10^9$ 'dur.
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- Hem $(0, S)$ hem de $(0, D)$ hücreleri bitki örtüsünden arındırılmıştır.

Alt Görevler

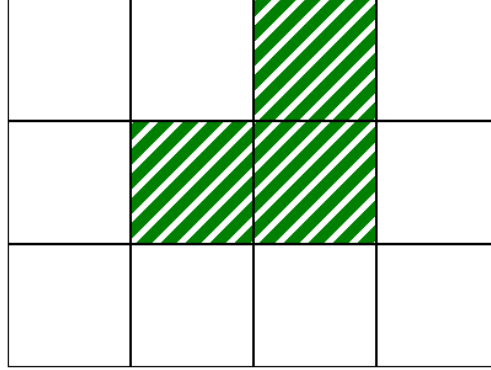
Alt Görev	Puan	Ek Kısıtlar
1	10	Her soru için $L = 0$ ve $R = M - 1$ 'dir. $N = 1$.
2	14	Her soru için $L = 0$ ve $R = M - 1$ 'dir. $1 \leq i < N$ şartını sağlayan her i için $T[i - 1] \leq T[i]$ 'dir.
3	13	Her soru için $L = 0$ ve $R = M - 1$ 'dir. $N = 3$ ve $T = [2, 1, 3]$.
4	21	Her soru için $L = 0$ ve $R = M - 1$ 'dir. $Q \leq 10$.
5	25	Her soru için $L = 0$ ve $R = M - 1$ 'dir.
6	17	Başka kısıt yoktur.

Örnek

Aşağıdaki çağırışı göz önüne alın.

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

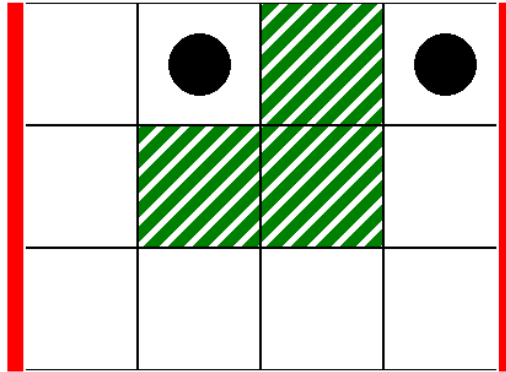
Bu, aşağıdaki görüntüdeki haritaya karşılık gelir; burada beyaz hücreler bitki örtüsünden arındırılmıştır.



İlk soru olarak aşağıdaki çağrıyı ele alalım.

```
can_reach(0, 3, 1, 3)
```

Bu, aşağıdaki görüntüdeki senaryoya karşılık gelir; burada kalın dikey çizgiler $L = 0$ ile $R = 3$ arasındaki sütun aralığını, siyah diskler ise başlangıç ve bitiş hücrelerini gösterir.



Bu durumda lama, $(0, 1)$ hücresinden $(0, 3)$ hücresine aşağıdaki geçerli yolu kullanarak ulaşabilir.

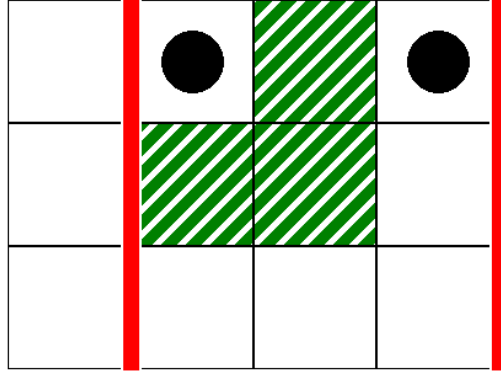
$(0, 1), (0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)$

Bu nedenle bu çağrı `true` değerini dönmelidir.

İkinci soru olarak şu çağrıyı ele alalım.

```
can_reach(1, 3, 1, 3)
```

Bu, aşağıdaki görseldeki senaryoya karşılık gelir.



Bu durumda, (0,1) hücresinden (0,3) hücresine, yoldaki tüm hücrelerin 1 ile 3 sütunları arasında (sınırlar dahil) yer aldığı geçerli bir yol yoktur. Bu nedenle bu çağrı `false` değerini dönmelidir.

Örnek Grader

Girdi formatı:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Burada, $L[k]$, $R[k]$, $S[k]$ ve $D[k]$ ($0 \leq k < Q$) her bir `can_reach` çağrısının parametrelerini belirtir.

Çıktı formatı:

```
A[0]
A[1]
...
A[Q-1]
```

Burada `can_reach(L[k], R[k], S[k], D[k])` çağrısı `true` dönüyorsa ($0 \leq k < Q$ olmak üzere) $A[k]$ 'nin değeri 1'dir; aksi taktirde $A[k]$ 'nin değeri 0'dir.